Experiment : 8

Query Processing on Large Datasets Using Apache Hive

Mapped CO(s) : CO2

Aim

To perform SQL-like querying, aggregation, and analysis of large datasets using Apache Hive.

Objectives

- **To understand the architecture and working of Apache Hive.**
- **To create Hive databases and tables.**
- **To load data into Hive tables.**
- **To perform SQL-like queries using HiveQL.**
- **To analyze data using filtering, sorting, grouping, and aggregation functions.**

Prerequisites

- **Basic knowledge of SQL.**
- **Basic understanding of Hadoop and HDFS.**
- **Apache Hadoop and Apache Hive installed and configured.**
- **Java Development Kit (JDK).**
- **Titanic Dataset stored in HDFS.**

Theory/Background

Apache Hive is an open-source data warehouse system built on top of Hadoop. It allows users to query and analyze large datasets stored in HDFS using HiveQL (Hive Query Language), which is similar to SQL.

Hive converts SQL-like queries into MapReduce, Tez, or Spark jobs, enabling efficient processing of large datasets without requiring users to write complex MapReduce programs.

Common Hive operations include:

- **CREATE DATABASE – Creates a new Hive database.**
- **CREATE TABLE – Creates a table in Hive.**
- **LOAD DATA – Loads data into a Hive table.**
- **SELECT – Retrieves data from a table.**
- **WHERE – Filters records based on a condition.**
- **GROUP BY – Groups records for aggregation.**
- **ORDER BY – Sorts the query results.**

- **COUNT(), AVG(), MAX(), MIN() – Aggregate functions used for analysis.**

Apache Hive simplifies big data analysis by providing a familiar SQL interface for querying massive datasets.

Algorithm/Methodology

- **Start Hadoop and Hive services.**
- **Create a Hive database.**
- **Create a table for the Titanic dataset.**
- **Load the dataset into the Hive table.**
- **Display the table contents.**
- **Execute SQL-like queries for filtering and aggregation.**
- **Group and sort the data.**
- **Analyze the query results.**

Dataset Description

Dataset Name: Titanic Dataset

File Name: Titanic-Dataset.csv

Description:

The Titanic dataset contains passenger information such as Passenger ID, Survival Status, Passenger Class, Name, Gender, Age, Fare, Cabin, and Embarked Port. The dataset is used to demonstrate SQL-like query processing and data analysis using Apache Hive.

Important Attributes

Attribute	Description
PassengerId	Unique passenger identifier
Survived	Survival status (0 = No, 1 = Yes)
Pclass	Passenger class
Name	Passenger name
Sex	Gender
Age	Passenger age
Fare	Ticket fare
Cabin	Cabin number
Embarked	Port of embarkation

Program/Implementation

The implementation should include:

- **Creating a database.**
- **Creating a Hive table.**
- **Loading the Titanic dataset.**
- **Running SELECT queries.**
- **Performing filtering, grouping, ordering, and aggregation.**

Test Cases

Test Case	Expected Result
Create database	Database created successfully
Create table	Table created successfully
Load dataset	Data loaded into Hive table
Execute SELECT query	Records displayed correctly
Group by passenger class	Aggregated results displayed
Calculate average fare	Average fare displayed

Experimental Results

Observation

- **The Hive database and table were created successfully.**
- **The Titanic dataset was loaded into the Hive table.**
- **SQL-like queries were executed successfully.**
- **Aggregation functions such as COUNT and AVG produced the expected results.**
- **Data was filtered, grouped, and sorted correctly using HiveQL.**

Result

Apache Hive successfully processed and analyzed the Titanic dataset using SQL-like queries. Query execution and aggregation operations demonstrated Hive's capability to analyze large datasets efficiently.

Conclusion

The experiment demonstrated how Apache Hive provides a simple SQL-like interface for querying and analyzing large datasets stored in Hadoop. HiveQL made it easy to perform filtering, grouping, sorting, and aggregation operations, making Hive an effective tool for big data analytics.

Exercises/Assignments

1. **Display only female passengers from the Titanic dataset.**
2. **Find the average fare for each passenger class.**
3. **Count the number of passengers who survived.**
4. **Sort passengers by age in descending order.**
5. **Find the maximum and minimum fare using HiveQL.**